

Corso Python + Flask

Sviluppo Gestionale con Sistema Multi-Utenza

Durata	80 ore (10 giorni x 8 ore)
Progetto	Gestionale Magazzino/Ordini
Ruoli Utente	Cliente, Fornitore, Amministratore

INDICE GIORNATE

Giorno 1: Setup Ambiente e Primo Progetto Flask

Giorno 2: Database con Flask-SQLAlchemy

Giorno 3: Sistema Utenti con Flask-Login

Giorno 4: Dashboard e Protezione Route

Giorno 5: Gestione Prodotti CRUD

Giorno 6: Carrello con Sessioni Flask

Giorno 7: Sistema Ordini

Giorno 8: Gestione Ordini e Notifiche

Giorno 9: Sistema Pagamenti con Stripe

Giorno 10: Deploy e Progetto Finale

MODALITA' DI LAVORO

La modalità di lavoro è strutturata in AGILE. L'AGILE è una modalità di lavoro che è suddivisa generalmente in sprint, gli sprint corrispondono ad una tempistica di tempo (Generalmente due settimane). In ogni sprint vengono suddivisi i vari task tra i developer scelti. La board o lavagna che compone o presenta questi task generalmente è divisa in Backlog/To Do, In Progress, Done .

Potete strutturare il vostro lavoro seguendo questa idea (usata nel mondo del lavoro tutt'oggi) con software quali Jira e GitLab.

GIORNO 1

Setup Ambiente e Primo Progetto Flask

TASK 1 - SEMPLICE

Creazione Ambiente Virtuale e Installazione Flask

Crea un virtual environment chiamato 'gestionale_env', attivalo, installa Flask e verifica che l'installazione sia corretta.

HINTS

Usa `python -m venv gestionale_env` o semplicemente `venv <nome_gestionale>`

Windows: `gestionale_env\Scripts\activate`

Linux/Mac: `source gestionale_env/bin/activate`

`pip install flask`

Verifica: `python -c "import flask; print(flask.__version__)"`

SNIPPET

```
# Comandi terminale
python -m venv gestionale_env
gestionale_env\Scripts\activate # Windows
source gestionale_env/bin/activate # Linux/Mac
pip install flask
```

```
# app.py - Test
from flask import Flask
app = Flask(__name__)

@app.route('/')
def home():
    return 'Flask funziona!'
```

```
if __name__ == '__main__':
    app.run(debug=True)
```

LINK UTILI

- Installazione Flask: <https://flask.palletsprojects.com/en/3.0.x/installation/>

Deliverable: Browser con 'Flask funziona!' su localhost:5000

TASK 2 - MEDIO

Struttura Progetto Flask

Organizza il progetto con struttura professionale: cartelle templates, static, package app, file di configurazione.

HINTS

Crea cartella app/ per il package

Dentro app: `__init__.py`, `routes.py`, `models.py`

`templates/` e `static/` dentro app

run.py nella root

SNIPPET

```
# Struttura:

# gestionale/
# └── app/
#     ├── __init__.py
#     ├── routes.py
#     ├── templates/
#     └── static/
# └── config.py
# └── run.py

# app/__init__.py
from flask import Flask

def create_app():
    app = Flask(__name__)
    app.config['SECRET_KEY'] = 'chiave-segreta'

    from app.routes import main
    app.register_blueprint(main)
    return app

# app/routes.py
from flask import Blueprint, render_template
main = Blueprint('main', __name__)

@main.route('/')
def home():
    return render_template('home.html')

# run.py
from app import create_app
app = create_app()
if __name__ == '__main__':
    app.run(debug=True)
```

LINK UTILI

- Application Factory: <https://flask.palletsprojects.com/en/3.0.x/patterns/appfactories/>

Deliverable: Progetto con struttura organizzata

TASK 3 - DIFFICILE

Template Base con Jinja2

Crea template base.html con ereditarietà Jinja2, header, footer e blocco content. Crea home.html che estende il base.

HINTS

{% block nome %}{% endblock %} per i blocchi
{% extends 'base.html' %} per ereditare
url_for('static', filename='...') per CSS
Passa variabili con render_template

SNIPPET

```
# templates/base.html
<!DOCTYPE html>
<html lang="it">
<head>
    <title>{% block title %}Gestionale{% endblock %}</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>
    <nav><a href="{{ url_for('main.home') }}">Home</a></nav>

    {% with messages = get_flashed_messages() %}
        {% for msg in messages %}<div class="alert">{{ msg }}</div>{% endfor %}
    {% endwith %}

    {% block content %}{% endblock %}
</body>
</html>

# templates/home.html
{% extends 'base.html' %}
{% block content %}
<h1>Benvenuto in {{ titolo }}</h1>
{% endblock %}
```

LINK UTILI

- Template Jinja2: <https://flask.palletsprojects.com/en/3.0.x/templating/>

Deliverable: Homepage con template inheritance

GIORNO 2

Database con Flask-SQLAlchemy

TASK 1 - SEMPLICE

Setup SQLAlchemy e Modello Categoria

Installa Flask-SQLAlchemy, configura SQLite e crea modello Categoria con: nome, descrizione, attiva.

HINTS

```
pip install flask-sqlalchemy
Configura SQLALCHEMY_DATABASE_URI
db = SQLAlchemy()
db.create_all() per creare tabelle
```

SNIPPET

```
# config.py

import os
basedir = os.path.abspath(os.path.dirname(__file__))

class Config:
    SECRET_KEY = 'chiave-segreta'
    SQLALCHEMY_DATABASE_URI = 'sqlite:/// ' + os.path.join(basedir, 'gestionale.db')
    SQLALCHEMY_TRACK_MODIFICATIONS = False

# app/__init__.py
from flask_sqlalchemy import SQLAlchemy
db = SQLAlchemy()

def create_app():
    app = Flask(__name__)
    app.config.from_object('config.Config')
    db.init_app(app)
    with app.app_context():
        db.create_all()
    return app

# app/models.py

from app import db

class Categoria(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    nome = db.Column(db.String(100), nullable=False)
    descrizione = db.Column(db.Text)
    attiva = db.Column(db.Boolean, default=True)
```

LINK UTILI

- Flask-SQLAlchemy: <https://flask-sqlalchemy.palletsprojects.com/>

Deliverable: Database con tabella Categoria

TASK 2 - MEDIO

Modello Prodotto con Relazioni

Crea modello Prodotto collegato a Categoria: codice, nome, descrizione, prezzo, quantita, categoria_id (FK), attivo.

HINTS

ForeignKey collega a Categoria
db.relationship per accesso bidirezionale
backref crea riferimento inverso
Numeric per prezzi

SNIPPET

```
# app/models.py

class Prodotto(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    codice = db.Column(db.String(50), unique=True, nullable=False)
    nome = db.Column(db.String(200), nullable=False)
    descrizione = db.Column(db.Text)
    prezzo = db.Column(db.Numeric(10, 2), nullable=False)
    quantita = db.Column(db.Integer, default=0)
    attivo = db.Column(db.Boolean, default=True)
    categoria_id = db.Column(db.Integer, db.ForeignKey('categoria.id'))
    categoria = db.relationship('Categoria', backref='prodotti')

# Flask shell

>>> from app.models import Categoria, Prodotto
>>> cat = Categoria(nome='Elettronica')
>>> db.session.add(cat)
>>> db.session.commit()
>>> prod = Prodotto(codice='P001', nome='Mouse', prezzo=29.99, categoria=cat)
>>> db.session.add(prod)
>>> db.session.commit()
```

LINK UTILI

- Relationships: <https://flask-sqlalchemy.palletsprojects.com/en/3.1.x/models/>

Deliverable: Prodotto collegato a Categoria

TASK 3 - DIFFICILE

Modello Fornitore con Many-to-Many

Crea modello Fornitore con: ragione_sociale, partita_iva, email, telefono. Collegalo ai Prodotti con relazione Many-to-Many.

HINTS

Crea tabella associazione con db.Table
Usa secondary nella relationship
Due ForeignKey nella tabella associazione

SNIPPET

```
# app/models.py

fornitore_prodotto = db.Table('fornitore_prodotto',
    db.Column('fornitore_id', db.Integer, db.ForeignKey('fornitore.id'), primary_key=True),
    db.Column('prodotto_id', db.Integer, db.ForeignKey('prodotto.id'), primary_key=True)
)

class Fornitore(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    ragione_sociale = db.Column(db.String(200), nullable=False)
    partita_iva = db.Column(db.String(11), unique=True)
    email = db.Column(db.String(120))
    telefono = db.Column(db.String(20))
    prodotti = db.relationship('Prodotto', secondary=fornitore_prodotto,
                               backref=db.backref('fornitori', lazy='dynamic'))

# Utilizzo

>>> f = Fornitore(ragione_sociale='Tech SRL', partita_iva='12345678901')
>>> f.prodotti.append(prod)
>>> db.session.commit()
```

LINK UTILI

- Many-to-Many: <https://flask-sqlalchemy.palletsprojects.com/en/3.1.x/models/#many-to-many-relationships>

Deliverable: Fornitore con prodotti associati

GIORNO 3

Sistema Utenti con Flask-Login

TASK 1 - SEMPLICE

Modello Utente con Ruoli

Installa Flask-Login e crea modello User con: username, email, password_hash, ruolo (CLIENTE, FORNITORE, ADMIN).

HINTS

- pip install flask-login
- Eredita da UserMixin
- werkzeug.security per hash password
- Inizializza LoginManager

SNIPPET

```
# pip install flask-login

# app/models.py

from flask_login import UserMixin
from werkzeug.security import generate_password_hash, check_password_hash

class User(UserMixin, db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(80), unique=True, nullable=False)
    email = db.Column(db.String(120), unique=True, nullable=False)
    password_hash = db.Column(db.String(256))
    ruolo = db.Column(db.String(20), default='CLIENTE')

    def set_password(self, password):
        self.password_hash = generate_password_hash(password)

    def check_password(self, password):
        return check_password_hash(self.password_hash, password)

# app/__init__.py

from flask_login import LoginManager
login_manager = LoginManager()

def create_app():
    # ...
    login_manager.init_app(app)
    login_manager.login_view = 'auth.login'

    @login_manager.user_loader
    def load_user(id):
        return User.query.get(int(id))
```

LINK UTILI

- Flask-Login: <https://flask-login.readthedocs.io/>

TASK 2 - MEDIO

Registrazione Utente

Crea form e route di registrazione con Flask-WTF. L'utente sceglie se registrarsi come Cliente o Fornitore.

HINTS

pip install flask-wtf email-validator
Crea RegistrationForm
Usa wtforms validators
Blueprint auth per autenticazione

SNIPPET

```
# pip install flask-wtf email-validator

# app/forms.py

from flask_wtf import FlaskForm
from wtforms import StringField, PasswordField, SelectField, SubmitField
from wtforms.validators import DataRequired, Email, EqualTo, Length

class RegistrationForm(FlaskForm):
    username = StringField('Username', validators=[DataRequired(), Length(min=3)])
    email = StringField('Email', validators=[DataRequired(), Email()])
    password = PasswordField('Password', validators=[DataRequired(), Length(min=6)])
    password2 = PasswordField('Conferma', validators=[EqualTo('password')])
    ruolo = SelectField('Tipo', choices=[('CLIENTE', 'Cliente'), ('FORNITORE', 'Fornitore')])
    submit = SubmitField('Registrati')

# app/auth/routes.py

from flask import Blueprint, render_template, redirect, url_for, flash
auth = Blueprint('auth', __name__)

@auth.route('/register', methods=['GET', 'POST'])
def register():
    form = RegistrationForm()
    if form.validate_on_submit():
        user = User(username=form.username.data, email=form.email.data, ruolo=form.ruolo.data)
        user.set_password(form.password.data)
        db.session.add(user)
        db.session.commit()
        flash('Registrazione completata!')
        return redirect(url_for('auth.login'))
    return render_template('auth/register.html', form=form)
```

LINK UTILI

- Flask-WTF: <https://flask-wtf.readthedocs.io/>

TASK 3 - DIFFICILE

Login e Redirect per Ruolo

Implementa login con Flask-Login. Dopo il login, redirect a dashboard diversa in base al ruolo.

HINTS

- login_user() per login
- current_user per utente corrente
- Controlla ruolo per redirect
- logout_user() per logout

SNIPPET

```
# app/forms.py

class LoginForm(FlaskForm):
    username = StringField('Username', validators=[DataRequired()])
    password = PasswordField('Password', validators=[DataRequired()])
    submit = SubmitField('Accedi')

# app/auth/routes.py

from flask_login import login_user, logout_user, current_user

@auth.route('/login', methods=['GET', 'POST'])
def login():
    if current_user.is_authenticated:
        return redirect(url_for('main.home'))

    form = LoginForm()
    if form.validate_on_submit():
        user = User.query.filter_by(username=form.username.data).first()
        if user and user.check_password(form.password.data):
            login_user(user)
            if user.ruolo == 'ADMIN':
                return redirect(url_for('main.dashboard_admin'))
            elif user.ruolo == 'FORNITORE':
                return redirect(url_for('main.dashboard_fornitore'))
            return redirect(url_for('main.dashboard_cliente'))
        flash('Credenziali non valide')
    return render_template('auth/login.html', form=form)

@auth.route('/logout')
def logout():
    logout_user()
    return redirect(url_for('main.home'))
```

LINK UTILI

- Login Example: <https://flask-login.readthedocs.io/en/latest/#login-example>

Deliverable: Login con redirect per ruolo

GIORNO 4

Dashboard e Protezione Route

TASK 1 - SEMPLICE

Dashboard Cliente

Crea dashboard Cliente con messaggio di benvenuto e lista prodotti disponibili. Proteggi con `@login_required`.

HINTS

- `@login_required` blocca utenti non loggati
- `current_user.username` per nome
- Filtra prodotti attivi
- `render_template` con context

SNIPPET

```
# app/routes.py
from flask_login import login_required, current_user

@main.route('/dashboard/cliente')
@login_required
def dashboard_cliente():
    if current_user.ruolo != 'CLIENTE':
        flash('Accesso non autorizzato')
        return redirect(url_for('main.home'))

    prodotti = Prodotto.query.filter_by(attivo=True).all()
    return render_template('dashboard/cliente.html', prodotti=prodotti)

# templates/dashboard/cliente.html

{% extends 'base.html' %}
{% block content %}
<h1>Ciao {{ current_user.username }}!</h1>
<h2>Prodotti Disponibili</h2>
{% for prodotto in prodotti %}
    <div class="prodotto">
        <h3>{{ prodotto.nome }}</h3>
        <p>€{{ prodotto.prezzo }}</p>
    </div>
{% else %}
    <p>Nessun prodotto</p>
{% endfor %}
{% endblock %}
```

LINK UTILI

- `login_required`: https://flask-login.readthedocs.io/en/latest/#flask_login.login_required

Deliverable: Dashboard cliente con prodotti

TASK 2 - MEDIO

Dashboard Fornitore

Crea dashboard Fornitore che mostra i prodotti forniti e statistiche base (totale prodotti, totale pezzi).

HINTS

Aggiungi user_id FK a Fornitore
func.sum() per totali
Crea profilo Fornitore se non esiste

SNIPPET

```
# app/models.py - Aggiorna Fornitore

class Fornitore(db.Model):
    # ...
    user_id = db.Column(db.Integer, db.ForeignKey('user.id'), unique=True)
    user = db.relationship('User', backref=db.backref('fornitore', uselist=False))

# app/routes.py

from sqlalchemy import func

@main.route('/dashboard/fornitore')
@login_required
def dashboard_fornitore():
    if current_user.ruolo != 'FORNITORE':
        return redirect(url_for('main.home'))

    fornitore = current_user.fornitore
    if not fornitore:
        flash('Completa il profilo')
        return redirect(url_for('main.profilo_fornitore'))

    prodotti = fornitore.prodotti
    totale_pezzi = sum(p.quantita for p in prodotti)

    return render_template('dashboard/fornitore.html',
        fornitore=fornitore, prodotti=prodotti, totale_pezzi=totale_pezzi)
```

LINK UTILI

- SQLAlchemy func: <https://docs.sqlalchemy.org/en/20/core/functions.html>

Deliverable: Dashboard fornitore con statistiche

TASK 3 - DIFFICILE

Dashboard Admin con Statistiche

Crea dashboard Admin con: totale utenti per ruolo, prodotti con scorte basse (<10), ultimi 5 utenti.

HINTS

group_by per raggruppare
func.count() per conteggi
order_by desc per ordinamento
limit(5) per limitare

SNIPPET

```
# app/routes.py

from sqlalchemy import desc

@main.route('/dashboard/admin')
@login_required
def dashboard_admin():
    if current_user.ruolo != 'ADMIN':
        return redirect(url_for('main.home'))

    utenti_per_ruolo = db.session.query(
        User.ruolo, func.count(User.id).label('totale')
    ).group_by(User.ruolo).all()

    scorte_basse = Prodotto.query.filter(
        Prodotto.quantita < 10, Prodotto.attivo == True
    ).all()

    ultimi_utenti = User.query.order_by(desc(User.id)).limit(5).all()

    stats = {
        'totale_prodotti': Prodotto.query.count(),
        'prodotti_attivi': Prodotto.query.filter_by(attivo=True).count()
    }

    return render_template('dashboard/admin.html',
        utenti_per_ruolo=utenti_per_ruolo, scorte_basse=scorte_basse,
        ultimi_utenti=ultimi_utenti, stats=stats)
```

LINK UTILI

- Query API: <https://flask-sqlalchemy.palletsprojects.com/en/3.1.x/queries/>

Deliverable: Dashboard admin completa

GIORNO 5

Gestione Prodotti CRUD

● TASK 1 - SEMPLICE

Lista e Dettaglio Prodotto

Crea route per lista prodotti con paginazione e dettaglio singolo prodotto.

HINTS

paginate() per paginazione
get_or_404() per prodotto non trovato
request.args.get('page', 1, type=int)

SNIPPET

```
# app/routes.py

@main.route('/prodotti')
def prodotto_list():
    page = request.args.get('page', 1, type=int)
    prodotti = Prodotto.query.filter_by(attivo=True).paginate(page=page, per_page=10)
    return render_template('prodotti/list.html', prodotti=prodotti)

@main.route('/prodotti/<int:id>')
def prodotto_detail(id):
    prodotto = Prodotto.query.get_or_404(id)
    return render_template('prodotti/detail.html', prodotto=prodotto)

# Template paginazione

{% if prodotti.has_prev %}
    <a href="?page={{ prodotti.prev_num }}">Precedente</a>
{% endif %}
Pagina {{ prodotti.page }} di {{ prodotti.pages }}
{% if prodotti.has_next %}
    <a href="?page={{ prodotti.next_num }}">Successiva</a>
{% endif %}
```

LINK UTILI

- Pagination: <https://flask-sqlalchemy.palletsprojects.com/en/3.1.x/pagination/>

Deliverable: Lista paginata e pagina dettaglio

● TASK 2 - MEDIO

Creazione e Modifica Prodotto

Crea route per aggiungere e modificare prodotti. Solo Admin può accedere.

HINTS

Decoratore custom per admin
Stesso form per create/update

obj=prodotto per edit
populate_obj per salvare

SNIPPET

```
# app/decorators.py

from functools import wraps

def admin_required(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        if not current_user.is_authenticated or current_user.ruolo != 'ADMIN':
            flash('Accesso riservato admin')
            return redirect(url_for('main.home'))
        return f(*args, **kwargs)
    return decorated

# app/routes.py

@main.route('/prodotti/nuovo', methods=['GET', 'POST'])
@login_required
@admin_required
def prodotto_create():
    form = ProdottoForm()
    if form.validate_on_submit():
        prodotto = Prodotto(codice=form.codice.data, nome=form.nome.data,
                             prezzo=form.prezzo.data, categoria_id=form.categoria.data.id)
        db.session.add(prodotto)
        db.session.commit()
        return redirect(url_for('main.prodotto_list'))
    return render_template('prodotti/form.html', form=form)

@main.route('/prodotti/<int:id>/modifica', methods=['GET', 'POST'])
@login_required
@admin_required
def prodotto_update(id):
    prodotto = Prodotto.query.get_or_404(id)
    form = ProdottoForm(obj=prodotto)
    if form.validate_on_submit():
        form.populate_obj(prodotto)
        db.session.commit()
        return redirect(url_for('main.prodotto_detail', id=id))
    return render_template('prodotti/form.html', form=form)
```

LINK UTILI

- Custom Decorators: <https://flask.palletsprojects.com/en/3.0.x/patterns/viewdecorators/>

Deliverable: Form creazione/modifica prodotti

TASK 3 - DIFFICILE

Eliminazione e Filtri Prodotti

Aggiungi eliminazione prodotti e filtri per categoria e ricerca per nome.

HINTS

POST per eliminazione
request.args per filtri GET

ilike per ricerca case-insensitive

db.or_ per combinare filtri

SNIPPET

```
# app/routes.py

@main.route('/prodotti/<int:id>/elimina', methods=['POST'])
@login_required
@admin_required
def prodotto_delete(id):
    prodotto = Prodotto.query.get_or_404(id)
    db.session.delete(prodotto)
    db.session.commit()
    flash('Prodotto eliminato')
    return redirect(url_for('main.prodotto_list'))

@main.route('/prodotti')
def prodotto_list():
    page = request.args.get('page', 1, type=int)
    categoria_id = request.args.get('categoria', type=int)
    search = request.args.get('q', '')

    query = Prodotto.query.filter_by(attivo=True)

    if categoria_id:
        query = query.filter_by(categoria_id=categoria_id)
    if search:
        query = query.filter(db.or_(
            Prodotto.nome.ilike(f'%{search}%'),
            Prodotto.codice.ilike(f'%{search}%')
        ))

    prodotti = query.paginate(page=page, per_page=10)
    categorie = Categoria.query.filter_by(attiva=True).all()
    return render_template('prodotti/list.html', prodotti=prodotti, categorie=categorie)
```

LINK UTILI

- Query Filters: <https://docs.sqlalchemy.org/en/20/orm/queryguide/query.html>

Deliverable: Lista con filtri e eliminazione

GIORNO 6

Carrello con Sessioni Flask

TASK 1 - SEMPLICE

Classe Carrello Base

Crea classe Cart che gestisce carrello con sessioni Flask. Implementa metodo aggiungi.

HINTS

session è un dizionario
session.modified = True dopo modifiche
Salva: {product_id: {qty, price}}
File cart.py dedicato

SNIPPET

```
# app/cart.py

from flask import session
from decimal import Decimal

class Cart:
    def __init__(self):
        if 'cart' not in session:
            session['cart'] = {}
        self.cart = session['cart']

    def add(self, prodotto, quantita=1):
        pid = str(prodotto.id)
        if pid not in self.cart:
            self.cart[pid] = {'qty': 0, 'price': str(prodotto.prezzo), 'nome': prodotto.nome}
        self.cart[pid]['qty'] += quantita
        self.save()

    def save(self):
        session.modified = True

    def __len__(self):
        return sum(item['qty'] for item in self.cart.values())

# app/routes.py

@main.route('/cart/add/<int:id>')
@login_required
def cart_add(id):
    prodotto = Prodotto.query.get_or_404(id)
    cart = Cart()
    cart.add(prodotto)
    flash(f'{prodotto.nome} aggiunto!')
    return redirect(request.referrer or url_for('main.prodotto_list'))
```

LINK UTILI

- Flask Sessions: <https://flask.palletsprojects.com/en/3.0.x/quickstart/#sessions>

Deliverable: Cart con funzione aggiungi

TASK 2 - MEDIO

Operazioni Carrello Complete

Estendi Cart con: rimuovi, modifica quantità, svuota, calcolo totale. Crea route corrispondenti.

HINTS

del self.cart[id] per rimuovere
session.pop('cart') per svuotare
Somma per get_total()
__iter__ per iterare

SNIPPET

```
# app/cart.py

class Cart:
    # ... precedenti ...

    def remove(self, prodotto_id):
        pid = str(prodotto_id)
        if pid in self.cart:
            del self.cart[pid]
            self.save()

    def update(self, prodotto_id, quantita):
        pid = str(prodotto_id)
        if pid in self.cart:
            if quantita > 0:
                self.cart[pid]['qty'] = quantita
            else:
                del self.cart[pid]
                self.save()

    def clear(self):
        session.pop('cart', None)

    def get_total(self):
        return sum(Decimal(item['price']) * item['qty'] for item in self.cart.values())

    def __iter__(self):
        for pid, item in self.cart.items():
            item['prodotto_id'] = pid
            item['totale'] = Decimal(item['price']) * item['qty']
            yield item

# Routes

@main.route('/cart')
def cart_detail():
    return render_template('cart/detail.html', cart=Cart())

@main.route('/cart/remove/<int:id>')
def cart_remove(id):
    Cart().remove(id)
    return redirect(url_for('main.cart_detail'))
```

LINK UTILI

- Decimal: <https://docs.python.org/3/library/decimal.html>

Deliverable: Carrello completo

TASK 3 - DIFFICILE

Pagina Carrello Interattiva

Crea pagina carrello con modifica quantità, validazione disponibilità e totale.

HINTS

Form POST per aggiornamento
Controlla disponibilità
flash per errori
input number con max

SNIPPET

```
# app/routes.py

@main.route('/cart', methods=['GET', 'POST'])
@login_required
def cart_detail():
    cart = Cart()

    if request.method == 'POST':
        for item in cart:
            qty_key = f"qty_{item['prodotto_id']}"
            if qty_key in request.form:
                new_qty = int(request.form[qty_key])
                prodotto = Prodotto.query.get(item['prodotto_id'])
                if prodotto and new_qty > prodotto.quantita:
                    flash(f'Solo {prodotto.quantita} disponibili per {prodotto.nome}')
                else:
                    cart.update(item['prodotto_id'], new_qty)
        return redirect(url_for('main.cart_detail'))

    cart_items = []
    for item in cart:
        prodotto = Prodotto.query.get(item['prodotto_id'])
        if prodotto:
            cart_items.append({'prodotto': prodotto, 'qty': item['qty'],
                               'totale': Decimal(item['price']) * item['qty']})

    return render_template('cart/detail.html', cart_items=cart_items, cart=cart)
```

LINK UTILI

- Flash Messages: <https://flask.palletsprojects.com/en/3.0.x/patterns/flashing/>

Deliverable: Carrello interattivo con validazione

GIORNO 7

Sistema Ordini

TASK 1 - SEMPLICE

Modelli Ordine

Crea modelli Ordine e OrdineDettaglio con stati PENDING, CONFERMATO, SPEDITO, CONSEGNATO.

HINTS

FK da Ordine a User (Foreign Key)
FK da Dettaglio a Ordine e Prodotto
default=datetime.utcnow
Property per subtotale

SNIPPET

```
# app/models.py
from datetime import datetime

class Ordine(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    cliente_id = db.Column(db.Integer, db.ForeignKey('user.id'))
    data_ordine = db.Column(db.DateTime, default=datetime.utcnow)
    stato = db.Column(db.String(20), default='PENDING')
    totale = db.Column(db.Numeric(10, 2), default=0)
    note = db.Column(db.Text)
    cliente = db.relationship('User', backref='ordini')
    dettagli = db.relationship('OrdineDettaglio', backref='ordine')

class OrdineDettaglio(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    ordine_id = db.Column(db.Integer, db.ForeignKey('ordine.id'))
    prodotto_id = db.Column(db.Integer, db.ForeignKey('prodotto.id'))
    quantita = db.Column(db.Integer)
    prezzo_unitario = db.Column(db.Numeric(10, 2))
    prodotto = db.relationship('Prodotto')

    @property
    def subtotale(self):
        return self.quantita * self.prezzo_unitario
```

LINK UTILI

- Relationships: <https://docs.sqlalchemy.org/en/20/orm/relationships.html>

Deliverable: Modelli Ordine funzionanti

TASK 2 - MEDIO

Checkout Base

Crea route checkout che converte carrello in ordine. Crea Ordine, dettagli e svuota carrello.

HINTS

Utente deve essere loggato
Cicla sul carrello
Salva prezzo corrente
db.session.flush() per ID

SNIPPET

```
# app/routes.py

@main.route('/checkout', methods=['GET', 'POST'])
@login_required
def checkout():
    cart = Cart()
    if len(cart) == 0:
        flash('Carrello vuoto')
        return redirect(url_for('main.cart_detail'))

    if request.method == 'POST':
        ordine = Ordine(cliente_id=current_user.id, totale=cart.get_total())
        db.session.add(ordine)
        db.session.flush()

        for item in cart:
            prodotto = Prodotto.query.get(item['prodotto_id'])
            if prodotto:
                det = OrdineDettaglio(ordine_id=ordine.id, prodotto_id=prodotto.id,
                                      quantita=item['qty'], prezzo_unitario=prodotto.prezzo)
                db.session.add(det)

        db.session.commit()
        cart.clear()
        flash(f'Ordine #{ordine.id} creato!')
        return redirect(url_for('main.ordine_detail', id=ordine.id))

    return render_template('cart/checkout.html', cart=cart)
```

LINK UTILI

- Session flush: https://docs.sqlalchemy.org/en/20/orm/session_api.html

Deliverable: Checkout che crea ordini

TASK 3 - DIFFICILE

Checkout con Verifica Disponibilità

Migliora checkout: verifica disponibilità, decrementa stock, gestisci errori con rollback.

HINTS

Controlla disponibilità prima
try/except per errori
db.session.rollback()
→ Aggiorna quantità prodotto

SNIPPET

```
# app/routes.py

@main.route('/checkout', methods=['GET', 'POST'])
@login_required
def checkout():
    cart = Cart()
    if len(cart) == 0:
        return redirect(url_for('main.cart_detail'))

    if request.method == 'POST':
        errori = []
        for item in cart:
            prodotto = Prodotto.query.get(item['prodotto_id'])
            if not prodotto or item['qty'] > prodotto.quantita:
                errori.append(f'{prodotto.nome}: non disponibile')

        if errori:
            for e in errori:
                flash(e)
            return redirect(url_for('main.cart_detail'))

        try:
            ordine = Ordine(cliente_id=current_user.id, totale=cart.get_total())
            db.session.add(ordine)
            db.session.flush()

            for item in cart:
                prodotto = Prodotto.query.get(item['prodotto_id'])
                det = OrdineDettaglio(ordine_id=ordine.id, prodotto_id=prodotto.id,
                                      quantita=item['qty'], prezzo_unitario=prodotto.prezzo)
                db.session.add(det)
                prodotto.quantita -= item['qty']

            db.session.commit()
            cart.clear()
            return redirect(url_for('main.ordine_detail', id=ordine.id))
        except:
            db.session.rollback()
            flash('Errore checkout')
            return redirect(url_for('main.cart_detail'))

    return render_template('cart/checkout.html', cart=cart)
```

LINK UTILI

- Rollback: https://docs.sqlalchemy.org/en/20/orm/session_api.html#sqlalchemy.orm.Session.rollback

Deliverable: Checkout sicuro con gestione errori

GIORNO 8

Gestione Ordini e Notifiche

● TASK 1 - SEMPLICE

Storico Ordini Cliente

Crea pagina dove cliente vede suoi ordini con data, stato, totale e dettaglio.

HINTS

- Filtra per `current_user.id`
- Ordina per data desc
- `joinedload` per ottimizzare
- Verifica proprietà ordine

SNIPPET

```
# app/routes.py
from sqlalchemy.orm import joinedload

@main.route('/ordini')
@login_required
def ordine_list():
    ordini = Ordine.query.filter_by(cliente_id=current_user.id)\
        .order_by(Ordine.data_ordine.desc()).all()
    return render_template('ordini/list.html', ordini=ordini)

@main.route('/ordini/<int:id>')
@login_required
def ordine_detail(id):
    ordine = Ordine.query.options(
        joinedload(Ordine.dettagli).joinedload(OrdineDettaglio.prodotto)
    ).get_or_404(id)

    if ordine.cliente_id != current_user.id and current_user.ruolo != 'ADMIN':
        flash('Accesso negato')
        return redirect(url_for('main.ordine_list'))

    return render_template('ordini/detail.html', ordine=ordine)
```

LINK UTILI

- Eager Loading: <https://docs.sqlalchemy.org/en/20/orm/queryguide/relationships.html>

Deliverable: Storico ordini funzionante

● TASK 2 - MEDIO

Gestione Ordini Admin

Crea pagina Admin per vedere tutti gli ordini, filtrare per stato e cambiare stato.

HINTS

Select per filtro stato
Form POST per cambio stato
Mostra info cliente
Paginazione

SNIPPET

```
# app/routes.py

@main.route('/admin/ordini')
@login_required
@admin_required
def admin_ordine_list():
    page = request.args.get('page', 1, type=int)
    stato = request.args.get('stato', '')

    query = Ordine.query.options(joinedload(Ordine.cliente))
    if stato:
        query = query.filter_by(stato=stato)

    ordini = query.order_by(Ordine.data_ordine.desc()).paginate(page=page, per_page=20)
    stati = ['PENDING', 'CONFERMATO', 'SPEDITO', 'CONSEGNATO', 'ANNULLATO']

    return render_template('admin/ordini.html', ordini=ordini, stati=stati)

@main.route('/admin/ordini/<int:id>/stato', methods=['POST'])
@login_required
@admin_required
def admin_cambia_stato(id):
    ordine = Ordine.query.get_or_404(id)
    nuovo_stato = request.form.get('stato')
    if nuovo_stato in ['PENDING', 'CONFERMATO', 'SPEDITO', 'CONSEGNATO', 'ANNULLATO']:
        ordine.stato = nuovo_stato
        db.session.commit()
        flash('Stato aggiornato')
    return redirect(url_for('main.admin_ordine_list'))
```

LINK UTILI

- select_related: <https://docs.sqlalchemy.org/en/20/orm/queryguide/relationships.html>

Deliverable: Pannello admin ordini

TASK 3 - DIFFICILE

Notifica Email Cambio Stato

Invia email al cliente quando stato ordine cambia. Usa Flask-Mail.

HINTS

pip install flask-mail
Configura MAIL settings
Funzione helper invio
Chiama dopo cambio stato

SNIPPET


```

# pip install flask-mail

# config.py

class Config:
    MAIL_SERVER = 'smtp.gmail.com'
    MAIL_PORT = 587
    MAIL_USE_TLS = True
    MAIL_USERNAME = 'tua@email.com'
    MAIL_PASSWORD = 'password'

# app/__init__.py

from flask_mail import Mail
mail = Mail()

def create_app():
    mail.init_app(app)

# app/email.py

from flask_mail import Message
from app import mail

def invia_notifica_ordine(ordine, vecchio_stato):
    msg = Message(f'Ordine #{ordine.id} - Aggiornamento',
        recipients=[ordine.cliente.email])
    msg.body = f'''Ciao {ordine.cliente.username},

Ordine #{ordine.id} aggiornato.
Stato: {vecchio_stato} -> {ordine.stato}'''

    try:
        mail.send(msg)
    except Exception as e:
        print(f'Errore email: {e}')

# Nella route admin_cambia_stato

vecchio_stato = ordine.stato
ordine.stato = nuovo_stato
db.session.commit()
invia_notifica_ordine(ordine, vecchio_stato)

```

LINK UTILI

- Flask-Mail: <https://pythonhosted.org/Flask-Mail/>

Deliverable: Email automatiche cambio stato

GIORNO 9

Sistema Pagamenti con Stripe

● TASK 1 - SEMPLICE

Setup Stripe

Registrati su Stripe, ottieni chiavi API test. Installa libreria e configura.

HINTS

stripe.com per account
Dashboard > API Keys
pip install stripe
Chiavi in config.py

SNIPPET

```
# pip install stripe

# config.py

class Config:
    STRIPE_PUBLIC_KEY = 'pk_test_...'
    STRIPE_SECRET_KEY = 'sk_test_...'

# app/__init__.py

import stripe

def create_app():
    stripe.api_key = app.config['STRIPE_SECRET_KEY']

# Test route

@main.route('/test-stripe')
def test_stripe():
    try:
        intent = stripe.PaymentIntent.create(amount=1000, currency='eur')
        return {'status': 'ok', 'client_secret': intent.client_secret}
    except stripe.error.StripeError as e:
        return {'error': str(e)}
```

LINK UTILI

- Stripe Dashboard: <https://dashboard.stripe.com/test/apikeys>

Deliverable: Stripe configurato e test funzionante

● TASK 2 - MEDIO

Pagamento con Stripe Checkout

Integra Stripe Checkout: redirect a Stripe per pagare, poi ritorno sul sito.

HINTS

Crea Session con line_items
success_url e cancel_url
metadata per ordine_id
Stripe gestisce form pagamento

SNIPPET

```
# app/routes.py

import stripe

@main.route('/pagamento/<int:ordine_id>')
@login_required
def create_checkout_session(ordine_id):
    ordine = Ordine.query.get_or_404(ordine_id)
    if ordine.cliente_id != current_user.id:
        return redirect(url_for('main.home'))

    line_items = []
    for det in ordine.dettagli:
        line_items.append({
            'price_data': {
                'currency': 'eur',
                'unit_amount': int(det.prezzo_unitario * 100),
                'product_data': {'name': det.prodotto.nome},
            },
            'quantity': det.quantita,
        })

    session = stripe.checkout.Session.create(
        payment_method_types=['card'],
        line_items=line_items,
        mode='payment',
        success_url=url_for('main.pagamento_success', ordine_id=ordine.id, _external=True),
        cancel_url=url_for('main.pagamento_cancel', ordine_id=ordine.id, _external=True),
        metadata={'ordine_id': ordine.id}
    )
    return redirect(session.url)

@main.route('/pagamento/success/<int:ordine_id>')
def pagamento_success(ordine_id):
    ordine = Ordine.query.get_or_404(ordine_id)
    ordine.stato = 'CONFERMATO'
    db.session.commit()
    return render_template('pagamento/success.html', ordine=ordine)
```

LINK UTILI

- Stripe Checkout: <https://stripe.com/docs/checkout/quickstart>

Deliverable: Pagamento Stripe funzionante

TASK 3 - DIFFICILE

Webhook Stripe

Implementa webhook per notifiche Stripe. Verifica firma e aggiorna stato ordine.

HINTS

Endpoint senza CSRF
Verifica firma webhook
checkout.session.completed
Stripe CLI per test

SNIPPET

```
# config.py

STRIPE_WEBHOOK_SECRET = 'whsec_...'

# app/routes.py

@main.route('/webhook/stripe', methods=['POST'])
def stripe_webhook():
    payload = request.get_data()
    sig = request.headers.get('Stripe-Signature')

    try:
        event = stripe.Webhook.construct_event(
            payload, sig, current_app.config['STRIPE_WEBHOOK_SECRET'])
    except:
        return 'Error', 400

    if event['type'] == 'checkout.session.completed':
        session = event['data']['object']
        ordine_id = session['metadata'].get('ordine_id')
        if ordine_id:
            ordine = Ordine.query.get(int(ordine_id))
            if ordine and ordine.stato == 'PENDING':
                ordine.stato = 'CONFERMATO'
                db.session.commit()

    return 'OK', 200

# Test locale:

# stripe listen --forward-to localhost:5000/webhook/stripe
```

LINK UTILI

- Stripe Webhooks: <https://stripe.com/docs/webhooks>

Deliverable: Webhook funzionante

GIORNO 10

Deploy e Progetto Finale

TASK 1 - SEMPLICE

Preparazione al Deploy

Prepara per produzione: requirements.txt, impostazioni sicurezza, variabili ambiente.

HINTS

pip freeze > requirements.txt
DEBUG = False
Variabili ambiente per secrets
Config per ambienti diversi

SNIPPET

```
# pip freeze > requirements.txt

# config.py

import os

class Config:
    SECRET_KEY = os.environ.get('SECRET_KEY') or 'dev-key'
    SQLALCHEMY_DATABASE_URI = os.environ.get('DATABASE_URL') or 'sqlite:///app.db'

class ProductionConfig(Config):
    DEBUG = False

class DevelopmentConfig(Config):
    DEBUG = True

config = {'development': DevelopmentConfig, 'production': ProductionConfig}

# .env (non committare!)

SECRET_KEY=chiave-produzione
DATABASE_URL=mysql://user:pass@host/db

# .gitignore
.env
*.pyc
__pycache__
*.db
```

LINK UTILI

- Flask Config: <https://flask.palletsprojects.com/en/3.0.x/config/>

Deliverable: Progetto pronto per deploy

TASK 2 - MEDIO

Deploy su PythonAnywhere

Deploy su PythonAnywhere (gratuito). Configura database, static files e WSGI.

HINTS

- Account pythonanywhere.com
- Git clone del codice
- Virtualenv e dipendenze
- WSGI file config

SNIPPET

```
# PythonAnywhere Bash:
git clone https://github.com/user/gestionale-flask.git
cd gestionale-flask
mkvirtualenv --python=/usr/bin/python3.10 gestionale_env
pip install -r requirements.txt

# WSGI file
import sys, os
path = '/home/username/gestionale-flask'
if path not in sys.path:
    sys.path.append(path)

os.environ['SECRET_KEY'] = 'chiave-produzione'

from app import create_app
application = create_app('production')

# Web tab - Static files:
# URL: /static/
# Directory: /home/username/gestionale-flask/app/static

# Database MySQL dalla tab Databases
# flask shell -> db.create_all()
```

LINK UTILI

- PythonAnywhere Flask: <https://help.pythonanywhere.com/pages/Flask/>

Deliverable: Sito live accessibile

TASK 3 - DIFFICILE

Completamento Progetto Finale

Completa gestionale con feature a scelta: export CSV.

HINTS

- CSV: modulo csv Python
- Documenta

SNIPPET

```
# Export CSV
```

```
import csv
from io import StringIO
from flask import Response

@admin.route('/admin/ordini/export')
@admin_required
def export_ordini():
    output = StringIO()
    writer = csv.writer(output)
    writer.writerow(['ID', 'Cliente', 'Data', 'Stato', 'Totale'])

    for ordine in Ordine.query.all():
        writer.writerow([ordine.id, ordine.cliente.username,
                        ordine.data_ordine.strftime('%d/%m/%Y'), ordine.stato, ordine.totale])

    output.seek(0)
    return Response(output.getvalue(), mimetype='text/csv',
                    headers={'Content-Disposition': 'attachment; filename=ordini.csv'})
```

LINK UTILI

- Python CSV: <https://docs.python.org/3/library/csv.html>

Deliverable: Gestionale completo con feature extra